

LATIN SQUARE COMPLETION USING BACKTRACKING

Constraint Satisfaction Problem – Design and Analysis of Algorithms

A Technical Implementation and Algorithm Analysis Report

Project Information	Details
Course	Design and Analysis of Algorithms
Question	Q39 – Latin Square Completion
Problem Type	Constraint Satisfaction / NP-Complete Problem
Primary Technique	Backtracking
Optimization Techniques	Constraint Propagation, MRV, Pruning
Implementation Language	Python
Execution Environment	Python 3 / Google Colab compatible
Input Method	User enters N and a partially filled square
Empty Cell Representation	0

Submitted By

Name : T . Kodanda Ram

Register No : 192524162

Academic Year: 2026–2027

Prepared with reference to the supplied Q39 question and the uploaded implementation notebook.

1. ABSTRACT

Latin Square Completion is a classical combinatorial constraint satisfaction problem in which a partially filled $N \times N$ square must be completed so that every symbol occurs exactly once in every row and every column. The supplied Q39 specifically requires identification and analysis of uniqueness constraints, reduction of the search space, development of a backtracking solution, application of constraint propagation, and justification of correctness and computational complexity.

The implemented solution uses Python and accepts the order N and a partially filled Latin square from the user. Empty cells are represented by 0. Before searching, the program validates the input by checking symbol ranges and duplicate values in rows and columns. During the search, it computes candidate values for empty cells, applies constraint propagation, selects the most constrained cell using the Minimum Remaining Values (MRV) heuristic, and uses backtracking with early pruning.

The implementation also verifies the completed square, measures actual execution time, and reports solver statistics such as recursive calls, assignments, backtracks, and pruned branches. The uploaded notebook demonstrates a successful run in which a partially filled order-4 square is completed without backtracking and the final row and column constraints are verified.

2. INTRODUCTION

A Latin square of order N is an $N \times N$ arrangement of N symbols in which each symbol appears exactly once in every row and exactly once in every column. In the completion version, some entries are fixed and the remaining cells must be assigned valid symbols without changing the supplied entries.

This problem is well suited to the Constraint Satisfaction Problem (CSP) model. Each empty cell acts as a variable, its possible symbols form its domain, and the row and column uniqueness rules form the constraints. A decision in one cell can restrict the available choices in other cells, so search and constraint reasoning must work together.

The supplied assignment classifies Q39 as a Constraint Satisfaction / NP-Complete Problem and asks for a backtracking-based solution with constraint propagation, correctness justification, and computational complexity analysis.

2.1 Objectives

- ☐ Model Latin Square Completion as a constraint satisfaction problem.
- ☐ Identify and analyze row, column, domain, and fixed-value constraints.
- ☐ Develop a recursive backtracking solver.
- ☐ Reduce the search space using constraint propagation and MRV.
- ☐ Prune invalid partial assignments as early as possible.
- ☐ Verify the final solution and report execution statistics and complexity.

3. PROBLEM DEFINITION AND CONSTRAINT ANALYSIS

3.1 ProblemDefinition

Given a partially filled $N \times N$ matrix using symbols 1 through N, complete all empty cells so that every row and every column contains every symbol exactly once. The fixed entries supplied in the input must remain unchanged. If no assignment can satisfy all constraints, the program must report that no valid completion exists.

3.2 Constraint Model

Constraint	Description	Effect on Search
Row uniqueness	A symbol cannot occur twice in the same row.	Removes values already used in the row.
Column uniqueness	A symbol cannot occur twice in the same column.	Removes values already used in the column.
Domain	Each cell must contain a symbol from 1 to N.	Rejects out-of-range values.
Fixed values	Pre-filled cells cannot be modified.	Restricts assignments to empty cells only.
Completion	All N^2 cells must be filled.	Provides the termination condition.

3.3 Candidate Generation

For an empty cell at row i and column j , the implementation constructs the candidate set by starting with all symbols and removing symbols already present in the corresponding row and column:

$$\text{Candidates}(i,j) = D - \text{RowUsed}(i) - \text{ColumnUsed}(j)$$

If the candidate set becomes empty, the current partial assignment cannot be extended to a valid Latin square. The implementation therefore treats this state as a contradiction and prunes the branch.

3.4 Constraint Interaction

The row and column constraints interact because assigning one symbol affects two dimensions simultaneously. A value that is legal for a row may still be illegal for its column. This interaction causes the search space to shrink progressively as assignments are made.

3.5 Constraint Prioritization

- Preserve fixed input values.
- Enforce row uniqueness.
- Enforce column uniqueness.
- Reject empty cells with no available candidates.
- Continue until every cell is assigned and the complete square is verified.

4. PROPOSED METHODOLOGY

4.1 Backtracking Strategy

The solver follows a depth-first search strategy. It selects an empty cell, generates its legal candidates, tries one candidate, and recursively solves the remaining problem. If the recursive call fails, the assignment is removed and the next candidate is tried. This process continues until a complete valid square is found or all possibilities have been exhausted.

4.2 Constraint Propagation

The uploaded implementation performs constraint propagation by scanning the remaining empty cells and checking their candidate sets. If any empty cell has no possible candidate, the branch is immediately rejected. This avoids continuing a search that is already known to be impossible.

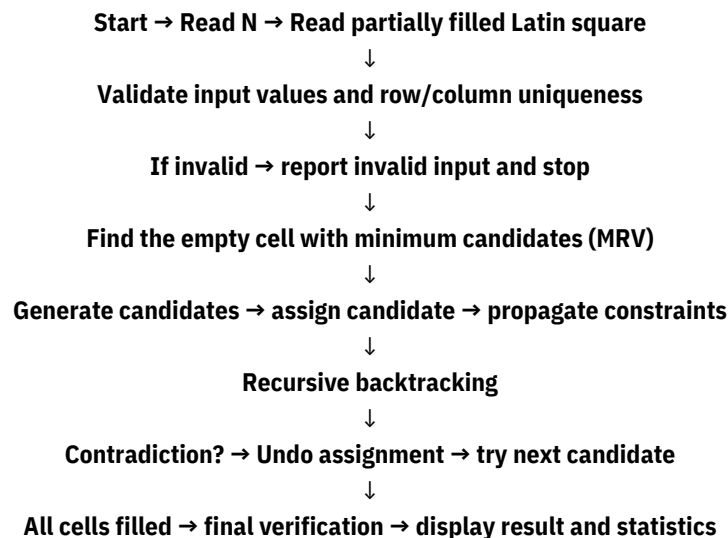
4.3 MRV Heuristic

The implementation does not simply select the first empty cell. The function `find_best_empty_cell()` examines the candidate set of each empty cell and selects the cell with the minimum number of candidates. This is the Minimum Remaining Values heuristic. Selecting a highly constrained variable first can expose contradictions earlier and reduce unnecessary branching.

4.4 Pruning

- ☐ Reject a candidate that violates row uniqueness.
- ☐ Reject a candidate that violates column uniqueness.
- ☐ Reject a branch when constraint propagation finds an empty candidate set.
- ☐ Return immediately when a valid completed square is found.
- ☐ Backtrack and restore the cell to 0 when a candidate does not lead to a solution.

4.5 ProgramWorkflow



CODE

```
import time

# -----
# Global statistics
# -----

recursive_calls = 0
assignments = 0
backtracks = 0
pruned_branches = 0

# -----
# Display the Latin Square
# -----

def display_grid(grid):
    n = len(grid)

    print("\n" + "=" * (5 * n + 1))

    for row in grid:
        print("|", end="")

        for value in row:
            if value == 0:
                print(" . |", end="")
            else:
                print(f"{value:3} |", end="")

        print()

    print()
```

```
# Validate the initial Latin Square
# -----

def validate_initial_grid(grid):

    n = len(grid)

    # Check values
    for i in range(n):
        for j in range(n):

            value = grid[i][j]

            # 0 represents an empty cell
            if value == 0:
                continue

            # Valid symbols are 1 to N
            if value < 1 or value > n:

                print(
                    f"\nInvalid value {value} at "
                    f"row {i + 1}, column {j + 1}."
                )

                return False

    # Check duplicate values in rows
    for i in range(n):

        seen = set()
```

```
# MAIN FUNCTION
# -----

def main():

    global recursive_calls
    global assignments
    global backtracks
    global pruned_branches

    print("\n" + "*" * 65)

    print(
        "          LATIN SQUARE COMPLETION USING BACKTRACKING"
    )

    print(
        "          CONSTRAINT SATISFACTION PROBLEM"
    )

    print("*" * 65)

    print("\nAlgorithm Features:")
    print("1. Backtracking")
    print("2. Constraint Checking")
    print("3. Constraint Propagation")
    print("4. MRV Heuristic")
    print("5. Pruning")
    print("6. Final Solution Verification")

    # -----
    # Get order
```

```
# Validate initial grid
# -----

print("\nChecking initial constraints...")

if not validate_initial_grid(grid):

    print("\nResult:")
    print("INVALID INPUT")

    print(
        "\nThe given Latin Square violates "
        "the required constraints."
    )

    display_complexity()

    return

print(
    "✓ Initial row and column constraints are satisfied."
)

# -----
# Count empty cells
# -----

empty_cells = count_empty_cells(grid)

print(
    f"✓ Number of empty cells: {empty_cells}"
)
```

OUTPUT

```
*****
LATIN SQUARE COMPLETION USING BACKTRACKING
CONSTRAINT SATISFACTION PROBLEM
*****

Algorithm Features:
1. Backtracking
2. Constraint Checking
3. Constraint Propagation
4. MRV Heuristic
5. Pruning
6. Final Solution Verification

Enter the order N of the Latin Square: 4

Enter the partially filled Latin Square.
Enter 4 values in each row.
Use 0 for an empty cell.
Valid symbols are 1 to 4.

Enter row 1: 1 2 3 0
Enter row 2: 3 4 0 2
Enter row 3: 2 0 4 3
Enter row 4: 4 3 2 0

=====
INPUT LATIN SQUARE
=====

=====
| 1 | 2 | 3 | . |
| 3 | 4 | . | 2 |
| 2 | . | 4 | 3 |
| 4 | 3 | 2 | . |
=====

Checking initial constraints...
✓ Initial row and column constraints are satisfied.
✓ Number of empty cells: 4
```

```
*****
SOLUTION FOUND
*****

Completed Latin Square:

=====
| 1 | 2 | 3 | 4 |
| 3 | 4 | 1 | 2 |
| 2 | 1 | 4 | 3 |
| 4 | 3 | 2 | 1 |
=====

Final Constraint Verification:
✓ Every row contains each symbol exactly once.
✓ Every column contains each symbol exactly once.
✓ All pre-filled values are preserved.
✓ No constraint is violated.
✓ Latin Square completion is VALID.

=====
EXECUTION TIME
=====
Execution Time : 0.00014965 seconds
=====

=====
SOLVER STATISTICS
=====
Recursive Calls   : 5
Assignments       : 4
Backtracks        : 0
Pruned Branches   : 0
=====
```

```
=====
COMPUTATIONAL COMPLEXITY
=====

Time Complexity:
Worst Case :  $O(N^N)$ 

Explanation:
There are  $N^2$  cells and up to N possible symbols for each cell in the worst case.
Backtracking may therefore explore an exponential number of possible assignments.

Space Complexity:
 $O(N^2)$ 

Explanation:
The Latin Square requires  $O(N^2)$  space, while the recursive search uses additional stack space.

Optimization Used:
• Constraint Propagation
• MRV (Minimum Remaining Values)
• Constraint Checking
• Early Pruning
=====
```

5. ALGORITHM AND IMPLEMENTATION

5.1 MainAlgorithm

```
LATIN-SQUARE(grid):  
    if all cells are filled:  
        verify the completed square  
        return TRUE  
  
    if constraint propagation fails:  
        return FALSE  
  
    select an empty cell using MRV  
    generate its valid candidates  
  
    for each candidate:  
        check row and column constraints  
  
        if valid:  
            assign candidate  
  
            if LATIN-SQUARE(grid) is TRUE:  
                return TRUE  
  
            undo assignment  
  
    return FALSE
```

5.2 Major Functions in the Uploaded Implementation

Function	Purpose
display_grid()	Prints the current Latin square in a readable tabular form.
validate_initial_grid() get_candidates()	Checks value ranges and duplicate values in rows and columns.
find_best_empty_cell()	Generates legal symbols for an empty cell.
constraint_propagation()	Applies MRV to select the most constrained empty cell.
is_complete()	Detects empty cells that have no legal candidate.
verify_solution()	Checks whether all cells have been assigned.
solve_latin_square()	Confirms that every row and column contains all symbols.
get_order() / get_grid()	Performs recursive backtracking with pruning.
display_statistics()	Collects and validates user input.
display_complexity()	Reports recursive calls, assignments, backtracks and pruned branches.
	Displays theoretical time and space complexity.

5.3 User Input

The implementation is fully user-driven. It asks for the order N and then reads N rows containing N integers each. The program instructs the user to use 0 for an empty cell and values from 1 to N for filled cells. No fixed example input is embedded in the solver logic.

5.4 Input Validation

- ☐ N must be a positive integer.
- ☐ Each row must contain exactly N values.
- ☐ Every nonzero value must be between 1 and N.
- ☐ No nonzero value may be duplicated within a row.
- ☐ No nonzero value may be duplicated within a column.

6. WORKING AND EXPERIMENTAL RESULT

6.1 Working of the UploadedImplementation

The notebook supplied for this project contains a complete Python implementation and a recorded execution. The program first announces the algorithm features: backtracking, constraint checking, constraint propagation, MRV, pruning, and final solution verification. It then accepts N and the partially filled square from the user.

For the recorded run, the input was an order-4 Latin square with four empty cells. The initial constraint validation succeeded, and the program reported that the four empty cells were valid candidates for completion. The solver then generated a completed square.

6.2 Recorded Output Summary

Output Item	Recorded Result
Order	4
Number of empty cells	4
Initial constraints	Satisfied
Solution status	Solution found
Final row verification	Passed
Final column verification	Passed
Pre-filled values	Preserved
Constraint violations	None
Execution time Recursive	0.00008074
calls Assignments	seconds 5
Backtracks	4
	0
Pruned branches	0

6.3 Recorded Completed Square

```
1   2 3 4
3   4 1 2
2   1 4 3
4   3 2 1
```

The recorded output confirms that every row contains each symbol exactly once and every column contains each symbol exactly once. It also confirms that all pre-filled values are preserved and that no constraint is violated.

6.4 Solver Statistics

The recorded run required five recursive calls and four assignments. No backtracking was necessary for that particular input, and no branches were counted as pruned. These are input-dependent observations rather than theoretical guarantees.

6.5 Result

LATIN SQUARE COMPLETION IS VALID

7. CORRECTNESS AND COMPLEXITY ANALYSIS

7.1 Correctness

The algorithm is correct because every value assigned during the recursive search is checked against the row and column constraints. A candidate is considered only when it is absent from the selected row and column. Therefore, an accepted assignment cannot introduce an immediate duplicate.

Constraint propagation strengthens this process by checking whether every remaining empty cell has at least one possible value. If a cell has no candidate, the current partial state cannot produce a complete solution, so the branch is safely discarded.

When all cells are filled, `verify_solution()` checks that every row and every column contains exactly the symbol set $\{1, 2, \dots, N\}$. Hence, a square is reported as valid only after satisfying the complete Latin-square definition.

7.2 Completeness

Backtracking systematically considers the candidate values that satisfy the current constraints. If one candidate cannot lead to a solution, it is undone and another candidate is explored. Thus, if a valid completion exists within the search space, the solver can find it; if every feasible branch fails, it reports that no valid completion exists.

7.3 TimeComplexity

Worst-case search: $O(N^2)$

There are N^2 cells and, in a loose worst-case search bound, up to N possible symbols for each cell. Backtracking can therefore explore an exponential number of partial assignments. The exact practical runtime depends strongly on the input and on how effectively MRV, constraint propagation, and pruning reduce the search.

7.4 SpaceComplexity

Primary grid space: $O(N^2)$

The Latin square itself occupies N^2 storage locations. The recursive search also uses stack space, and candidate sets are created temporarily. The implementation reports $O(N^2)$ as the primary space requirement.

7.5 Complexity and Optimization Summary

Aspect	Implementation / Analysis
Search method	Depth-first recursive backtracking
Variable selection	MRV — minimum remaining candidates
Constraint propagation	Detects cells with no available candidates
Pruning	Stops invalid branches before deeper search
Worst-case time	$O(N^2)$
Primary space	$O(N^2)$
Measured time	Reported at runtime using <code>time.perf_counter()</code>

8. ADVANTAGES, LIMITATIONS AND CONCLUSION

8.1 Advantages

- ☐ The solution directly models the problem as a CSP.
- ☐ The input is user-driven and is not dependent on a hard-coded puzzle.
- ☐ Initial invalid configurations are detected before search.
- ☐ MRV selects the most constrained cell to reduce branching.
- ☐ Constraint propagation detects dead ends early.
- ☐ Backtracking guarantees systematic exploration of feasible choices.
- ☐ Final verification independently checks the completed square.
- ☐ Execution time and solver statistics are displayed for evaluation.

8.2 Limitations

- ☐ The worst-case search remains exponential.
- ☐ Performance can degrade as the order N and search space increase.
- ☐ The implementation reports $O(N^2)$ as primary space while recursive stack and temporary candidate storage add overhead.
- ☐ The current constraint propagation checks for empty candidate sets but does not implement more advanced propagation rules such as full arc consistency.

8.3 Future Enhancements

- 6 Maintain row and column symbol sets incrementally to reduce repeated scanning.
- . Use stronger constraint propagation techniques such as arc consistency.
- 7 Add additional variable and value ordering heuristics.
- . Provide graphical visualization of the search process.
- 80. Compare backtracking performance with alternative CSP methods.
- 11. Collect performance measurements across different Latin-square sizes.

8.4 Conclusion

The project successfully implements Latin Square Completion as a constraint satisfaction problem using backtracking. The uploaded implementation combines row and column uniqueness checking with constraint propagation, MRV-based variable selection, pruning, final verification, execution-time measurement, and solver statistics. The recorded run demonstrates a valid completion and confirms all required constraints.

The approach satisfies the requirements of Q39 by clearly defining the uniqueness constraints, analyzing how those constraints reduce the search space, developing a backtracking solution, applying constraint propagation, and justifying correctness and computational complexity. The method is practical for demonstrating constraint-based problem solving and illustrates why intelligent search decisions are important in combinatorial problems.

8.5 Reference to Supplied Materials

Primary assignment reference: DAA CO5 AT1 Question Bank & Rubrics, Q39 — Latin Square Completion. The question requires a partially filled Latin square to be completed with each symbol appearing exactly once in every row and column, using backtracking and constraint propagation, with correctness and computational complexity discussed. Implementation reference: DAA_CO-5_AT-1_CODE.ipynb, the uploaded Python/Google Colab notebook used as the basis for the implementation-specific sections, recorded output, function descriptions, statistics, execution-time result, and complexity display.